

Comprehensive Code Explainer

In-Depth Walkthrough for DITEC Evaluation

Introduction

This document provides a detailed, line-by-line explanation of the core logic powering the AI Attendance Kiosk. It is designed to prepare you for any deep technical questions the evaluators might ask regarding the React frontend, the Node.js API, race conditions, and containerized deployments.

1 1. Frontend: State Machine & UI (pages/index.js)

The Kiosk frontend is built in Next.js (React). Because this is a physical kiosk, users cannot be allowed to freely click around. They must be guided through a strict sequence.

1.1 1.1 The State Machine

We control the flow using a React state variable called `step`.

State Definition

```
1 const [step, setStep] = useState('LOADING_MODELS');  
2 // Possible States:  
3 // 'LOADING_MODELS' -> 'SCANNING_QR' -> 'SCANNING_FACE' -> 'PROCESSING'  
  -> 'SUCCESS'
```

How it works:

- **LOADING_MODELS**: The app downloads the heavy `face-api.js` AI weights (like ResNet or MobileNet). The user sees a spinner.
- **SCANNING_QR**: The webcam activates. The `jsQR` library scans every frame looking for a QR code. Once found, we extract the `userId` and automatically transition to the next step.
- **SCANNING_FACE**: The webcam now looks for a face to verify the identity of the person holding the QR code.
- **PROCESSING**: The API is called to save the attendance.
- **SUCCESS**: The user sees the green success page, which auto-redirects back to **SCANNING_QR** after 4 seconds for the next person.

1.2 1.2 Debouncing the API Call

When the face is recognized, we need to call the backend. But face recognition runs at 60 FPS. It could easily fire 60 API requests in a single second!

API Debounce (isSubmittingRef)

```
1 const isSubmittingRef = useRef(false);  
2  
3 const markAttendance = async (userId, type) => {  
4   // If we are already sending a request, completely ignore this  
   trigger.  
5   if (isSubmittingRef.current) return;  
6  
7   isSubmittingRef.current = true; // Lock the frontend  
8   setStep('PROCESSING');  
9  
10  try {  
11    const response = await fetch('/api/attendance', { /* payload */ })  
12    ;  
13    if (response.ok) setStep('SUCCESS');  
  } finally {
```

```
14     isSubmittingRef.current = false; // Unlock after network request
15         finishes
16     }
17 };
```

What to explain: "By using a React `useRef`, I created a frontend lock. It guarantees that the frontend will only ever send exactly *one* HTTP request to the server per user interaction, protecting network bandwidth."

2. Backend: Mutex Locks & Validation (pages/api/attendan

Even with frontend protection, a malicious user or network glitch could bypass it. The backend *must* be the final source of truth.

2.1 In-Memory Mutex Lock

This is the fix for the Time-of-Check-to-Time-of-Use (TOCTOU) race condition.

Global Mutex Lock

```

1 // Defined OUTSIDE the handler so it persists across requests
2 const activeLocks = new Map();
3
4 export default async function handler(req, res) {
5   const { userId, type, isTest } = req.body;
6
7   // 1. Check if user is currently locked
8   if (activeLocks.has(userId)) {
9     return res.status(409).json({ error: 'Processing previous request.'
10    });
11  }
12
13  // 2. Lock the user
14  activeLocks.set(userId, true);
15
16  try {
17    // ... Database checks and writes go here ...
18  } finally {
19    // 3. ALWAYS release the lock, even if the database crashes
20    activeLocks.delete(userId);
21  }
22 }
```

What to explain: "I implemented a Hash Map (`activeLocks`) in Node.js memory. When a request comes in, we instantly flag that `userId` as 'Processing'. If a duplicate request arrives one millisecond later, it hits the `has()` check and is immediately thrown out with an HTTP 409 Conflict. Once the database finishes, the `finally` block deletes the lock so the user can punch out later."

2.2 Time Validation & Demo Mode

The system enforces strict 11 AM (In) and 5 PM (Out) constraints using the `date-fns-tz` library to ensure we always use Indian Standard Time (IST), regardless of where the server is hosted.

Time Validation

```

1 const TIMEZONE = process.env.APP_TIMEZONE || 'Asia/Kolkata';
2
3 // If we check the 'Demo Mode' box on the frontend, isTest is true.
4 if (!isTest) {
5   if (type === 'IN' && hours >= 11) {
6     return res.status(400).json({ error: 'Too late. Must be before
7       11:00 AM.' });
8   }
9 }
```

```
8  if (type === 'OUT' && hours < 17) {  
9      return res.status(400).json({ error: 'Too early. Must be after 5:00  
      PM.' });  
10 }  
11 }
```

3. Persistence: SQLite & Docker (lib/db.js)

By default, Docker containers are entirely stateless. If the server crashes or we deploy an update, the container is destroyed and recreated, meaning all attendance data would be lost.

3.1 Dynamic DB Pathing

To fix this, the app must know *where* to save the database.

lib/db.js

```
1 import path from 'path';
2
3 // If DB_PATH exists in environment, use it. Otherwise, use local
4 // folder.
5 const dbPath = process.env.DB_PATH || path.join(process.cwd(), '
6   attendance.db');
7
8 export const db = createClient({
9   url: 'file:' + dbPath,
10 });
```

3.2 Docker Volume Mapping

We use docker-compose.yml to map the path.

docker-compose.yml

```
1 services:
2   attendance-system:
3     build: .
4     volumes:
5       - ./data:/app/data # Maps host machine to container
6     environment:
7       - DB_PATH=/app/data/attendance.db
```

What to explain: "Inside the container, Node.js thinks it is saving the database to /app/data/attendance.db. However, Docker intercepts this and actually saves it to the ./data folder on the host computer's hard drive. Because of this, we can delete and rebuild the Docker container a thousand times, and the database will never be lost."